

Guía del Profesor -- Módulo I: Diseño Ágil + Python

Tema 01 | IF009 Laboratorio de Programación y Lenguajes 2026

Universidad Nacional de Tierra del Fuego -- Instituto IDEI

Matías Gel -- Docente titular

Docente

Prof. Matias Gel

2026

Índice rápido

- Resumen ejecutivo del módulo
 - Plan de clase detallado por sesión
 - Conceptos clave con ejemplos
 - Conexión curricular y evidencia académica
 - TP2 -- Análisis y rúbrica completa
 - Antipatrones frecuentes
 - Preguntas socráticas preparadas
 - Ruta a los recursos del módulo
-

1. Resumen Ejecutivo

¿De que se trata este módulo?

El Módulo I introduce a los alumnos al ciclo de desarrollo ágil con Python 3.13. No es solo Python como lenguaje -- es Python en el contexto del flujo de trabajo profesional: Git + CI + Codespaces + Ruff + Copilot.

Lo que el alumno SÍ debe saber al terminar

Habilidad	Dónde se evalúa
Funciones Python con type hints + docstring	Scripts `src/` del TP2
Usar colecciones (list, dict, set) con comprensiones	Scripts complejos del TP2
Push -> CI verde sin errores Ruff	Log de GitHub Actions
Documentar prompts IA en PROMPTS.md	PROMPTS.md del TP2

Lo que NO pertenece a este módulo (scope control)

Tema	Dónde va
pytest / TDD (escribir tests)	**Módulo II**
Clases y OOP	**Módulo IV**
async/await	Fuera del cuatrimestre
Django, modelos, ORM	**Módulos III-IV**
Type hints avanzados (Protocol, TypeVar)	Solo mención en Módulo IV

2. Plan de Clase

T1 -- Semana 2, Teoría (180 min)

```

00:00 - 00:45 T1-A: Modelo Ágil + VS Code + Codespaces
- Apertura: agenda del módulo (F-01)
- Manifiesto ágil: 4 valores (F-02, F-03)
- Cascada vs. ágil en tabla (F-04)
- Ciclo iterativo -> conexión con TP2 (F-05)
- Demo VS Code + extensiones (F-06, F-07, F-08, F-09)
- DevContainer JSON (F-10, F-11)
- Cierre socrático (F-12)

00:50 - 01:35 T1-B: Python 3.13 Fundamentos
- Por que Python en 2026 (F-13)
- Tipos primitivos + tabla (F-14)
- Asignación y operadores (F-15)
- Demo REPL 3.13 + errores contextuales (F-16, F-17) <- DEMO CLAVE
- Strings: inmutabilidad + metodos + f-strings (F-18, F-19, F-20)
- Referencias: el ejercicio de la lista compartida (F-21) <- EJERCICIO CLAVE
- None, bool peculiaridades (F-22)
- Conversiones de tipo (F-23)
- str vs list (F-24) + is/in (F-25)
- Cierre preguntas rápidas (F-26)

01:40 - 02:10 T1-C: Control de flujo + Funciones
- if/elif/else con función clasificar_nota (F-27)
- for sobre secuencias + antipatrón índice (F-28)
- while + break + continue (F-29)
- match PEP 636 (F-30)
- def + return + docstrings (F-31) <- IMPORTANTE PARA TP2
- Parámetros (F-32) + *args/**kwargs (F-33)
- Scope LEGB (F-34)
- Recursión básica (F-35)
- Cierre socrático es_primo (F-36)

02:15 - 02:35 T1-D: Ruff + PEP 8 + Preview TP2
- PEP 8: 5 puntos clave (F-37)
- Código autodocumentado (F-38)
- Convenciones nombres (F-39)
- Demo Ruff en vivo (F-40)
- Preview repo TP2 (F-41) + cierre (F-42)

```

P1 -- Semana 2, Práctica (180 min)

```

00:00 - 00:50 P1-A: Ejercicios guiados en Codespaces
- Ejercicio live: calculadora (match + type hints)
- Conversor temperatura (funciones puras)
- Fibonacci (while + list)
- Clasificador numerico (es_primo + match)

00:55 - 01:45 P1-B: Prompting para Python
- Demo prompt simple vs. RCTAE en ChatGPT
- Demo Copilot inline: entender antes de aceptar
- Ejercicio individual: PROMPTS.md primer draft

01:50 - 02:30 P1-C: GitHub Classroom TP2
- Aceptar TP2 (link F-41)
- Explorar repo + CI workflow
- Implementar hello.py -> push -> CI verde

02:35 - 02:55 P1-D: Revisión commits
- git log --oneline comparativo
- Convención conventional commits básica
- 2 commits por alumno con mensajes descriptivos

```

T2 -- Semana 3, Teoría (180 min)

```
00:00 - 00:45 T2-A: Colecciones Python
- Check progreso TP2 (encuesta rápida)
- Tabla comparativa 4 colecciones (F-44)
- list: slicing, sort, builtins (F-45)
- tuple: unpacking, hashable (F-46)
- dict: .items(), .get() (F-47)
- set: operaciones de conjuntos (F-48)
- Comprensiones: lista + dict + set (F-49, F-50)
- enumerate() + zip() (F-51)
- sorted(key=) + builtins (F-52, F-53)

00:50 - 01:35 T2-B: Módulos + HOF + Decoradores
- import, from...import, estructura TP2 (F-57, F-58)
- requirements.txt (F-59)
- HOF como concepto (F-60)
- map() + filter() (F-61, F-62)
- lambda (F-63) + sorted(key=lambda) (F-64)
- Decoradores: concepto + @property (F-65, F-66)
- @staticmethod, @functools.wraps (F-67, F-68)
- Aplicación en TP2 (F-69)

01:40 - 02:10 T2-C: Type Hints Python 3.10+
- ¿Que son? (F-71)
- PEP 484 -> sintaxis moderna (F-72)
- list[int], dict[str, float], tuple (F-73, F-74)
- X | None (F-75) + Union + Any (F-76, F-77)
- Callable (F-78)
- Ruff + Pylance verificando (F-79)

02:15 - 02:35 T2-D: Prompting para Debugging
- Traceback -> prompt RCTAE (F-81)
- Explicación línea a línea (F-82)
- Refactoring con IA (F-83)
- Cierre Módulo I + preview Módulo II (F-84)
```

P2 -- Semana 3, Práctica (180 min)

```
00:00 - 01:00 P2-A: Taller TP2 intensivo
- Meta: 5/7 scripts con CI verde al finalizar

01:05 - 01:45 P2-B: PROMPTS.md completo
- Revisión entre pares con checklist
- Meta: 3 prompts con comprensión línea a línea

01:50 - 02:30 P2-C: Revisión de commits
- git log --oneline --graph
- Meta: 7 commits descriptivos

02:35 - 02:55 P2-D: Retrospectiva + Preview Módulo II
- Pizarrón: Quedó claro / Costó mucho / Quiero saber +
- Red -> Green -> Refactor como preview TDD
```

3. Conceptos Clave

3.1 Manifiesto Ágil -- lo que tenes que transmitir

El punto central es el cambio de mentalidad: en cascada, el software se valida tarde y el cambio es costoso. En ágil, el software se valida continuamente y el cambio es bienvenido.

Analogía para la clase: Los tests del TP2 son "el cliente" del modelo ágil. No saben cómo vas a implementar la función -- solo que debe hacer. Tu ciclo de desarrollo es:

- Leer el test (conocer el requisito)
- Implementar (construir)
- Push -> CI (validar)
- Ajustar si falla (iterar)

Ese ciclo de 5 minutos es exactamente el corazón del desarrollo ágil.

3.2 Python 3.13 -- novedades a demostrar en clase

REPL mejorado:

```
python3
>>> def saludar(nombre: str) -> str:
...     """Saluda."""
...     return f"Hola {nombre}"
...
>>> saludar("Ada")
'Hola Ada'
```

bash

Mensajes de error contextuales:

```
>>> "hola".upper_case()
# Python 3.13:
AttributeError: 'str' object has no attribute 'upper_case'. Did you mean: 'upper'?
```

python

3.3 El ejercicio de referencias -- el más importante de T1

Este ejercicio produce el "momento aha" más potente del módulo:

```
a = [1, 2, 3]
b = a          # b NO es una copia -- apunta al mismo objeto
b.append(99)
print(a)        # [1, 2, 3, 99] <- muchos se sorprenden

# La solución pythónica
c = a.copy()    # 0: a[:] 0: list(a)
c.append(100)
print(a)        # [1, 2, 3, 99] -- sin cambios
```

python

Por que importa para el TP2: Si una función del TP2 modifica la lista que recibe como parámetro, el test puede fallar de formas inesperadas. Siempre copiar si vas a modificar.

3.4 Type hints -- lo que tienen que saber para el TP2

Regla mínima para el TP2:

```
# OBLIGATORIO en cada función pública
def mi_funcion(param1: tipo1, param2: tipo2 = valor_default) -> tipo_retorno:
    """Docstring obligatoria."""
    ...
```

python

Tabla de referencia rápida:

¿Que guardar?	Anotación	
Texto	`str`	
Número entero	`int`	
Número decimal	`float`	
Sí/No	`bool`	
Lista de strings	`list[str]`	
Lista de enteros	`list[int]`	
Dict string -> int	`dict[str, int]`	
Tupla de dos floats	`tuple[float, float]`	
String o nada	`str \`	None`
Sin retorno	`-> None`	

3.5 Comprensiones -- el patrón más pythónico

El patrón mental para transformar un for loop en comprensión:

```
LISTA: [f(x) for x in iterable if condición]
DICT:  {k: v for k, v in items() if condición}
SET:   {f(x) for x in iterable if condición}
```

Cuándo NO usar comprensión:

- Si la lógica tiene más de 2 condiciones
- Si necesitas múltiples pasos intermedios
- Si alguien que no conoce el código va a tener dificultad para leerla

3.6 PROMPTS.md -- el criterio de evaluación

El PROMPTS.md vale el 20% de la nota del TP2. Para aprobar, cada prompt necesita:

- Role -- quien es Copilot/ChatGPT para esta consulta
- Contexto -- que archivo, que función, que requisito
- Tarea -- que exactamente se le pide
- Restricciones -- que no puede hacer
- Comprensión línea a línea -- el alumno explica cada línea generada

El criterio de calidad: ¿Puede otro alumno leer el PROMPTS.md y reproducir exactamente el mismo proceso? Si la respuesta es no, el prompt es incompleto.

4. Conexión Curricular y Evidencia Académica

4.1 Mapa con el plan mínimo institucional

Este módulo cubre íntegramente el Módulo I del programa institucional:

Tópico plan-mínimo	Filminas	Sesión
El Modelo Ágil para construcción de aplicaciones	F-02 a F-05	T1-A
Herramientas de desarrollo ágil	F-06 a F-11	T1-A
Entornos de desarrollo integrado (IDE)	F-06 a F-09	T1-A
Código autodocumentado y herramientas de desarrollo	F-38 a F-40, F-79	T1-D, T2-C
Construcción de aplicaciones en entornos integrados	Sesiones P1 y P2	Prácticas

Pautas y criterios para la transformación de diseño	F-37 a F-41	T1-D
Lenguajes dinámicos para desarrollo de aplicaciones	F-13 (Python)	T1-B
Introducción al lenguaje Python (sintaxis, tipos)	F-13 a F-84	T1-B + T2-A

4.2 Decisiones pedagógicas basadas en evidencia

Decisión de diseño	Fuente académica	Descripción
Bloques de 45'/30'/20' en sesión T1	Karymsakova et al. (2025)	Ciclos cortos T->P mejoran retención
Copilot habilitado desde P1	Fan et al. (2025)	AI pair programming reduce ansiedad, mejora motivación
PROMPTS.md obligatorio	Alves & Cipriano (2024)	Previene uso de IA sin comprensión
IA literacy como hilo transversal	Prather (2024)	Marco pedagógico que aumenta uso crítico vs. ingenuo

4.3 Hilo conceptual del módulo

Tema 00 (Sem 1) -> Tema 01 (Sem 2-3) -> Tema 02 (Sem 4-5)
GitHub + Codespaces Ágil + Python TDD + pytest
(setup + TP1) (fundamentos + TP2) (testing + TP3)

Los alumnos que llegaron al Módulo I ya tienen cuenta GitHub activa, TP1 entregado y Codespace funcional. El puente natural es: "ya saben hacer commits y CI verde -- ahora aprenden el lenguaje para que el código sea bueno".

5. TP2 -- Análisis Completo

5.1 Estructura del repositorio

```
tp2-python-prompting-USUARIO/  
??? .devcontainer/  
?   ??? devcontainer.json      <- Python 3.13 + Ruff + Copilot  
??? .github/  
?   ??? workflows/  
?   ??? classroom.yml         <- ruff check + pytest  
??? src/  
?   ??? __init__.py  
?   ??? hello.py  
?   ??? calculadora.py  
?   ??? temperatura.py  
?   ??? fibonacci.py  
?   ??? colecciones.py  
?   ??? funciones_ord.py  
?   ??? type_hints.py  
??? tests/  
?   ??? __init__.py  
?   ??? test_hello.py  
?   ??? [un test por script]  
??? PROMPTS.md  
??? requirements.txt
```

5.2 Mapping scripts -> Módulo I

Script	Conceptos de T1	Conceptos de T2	Dificultad
`hello.py`	f-strings, type hints básico	--	???
`calculadora.py`	match, guard clauses, `float`	None`	--
`temperatura.py`	Funciones puras, type hint	--	???

`fibonacci.py`	while loop, list slicing	--	???
`coleccion.es.py`	list, dict	Comprensiones, dict ops	???
`funciones_ord.py`	Funciones de 1er clase	map(), filter(), lambda	???
`type_hints.py`	--	Todos los tipos avanzados	???

5.3 Rúbrica detallada

> Scripts (60%)

Criterio	Puntaje	Descripción
7/7 scripts con CI verde	60	Todos los tests de pytest pasan en GitHub Actions
6/7 scripts	50	--
5/7 scripts	40	**Mínimo aprobatorio**
4/7 o menos	0	Desaprobado en este componente
Código con Ruff clean	Requerido	`ruff check .` sin errores (CI lo verifica)
Docstrings en todas las funciones	Requerido	Pylance muestra warning, Ruff D100/D103
Type hints en todos los parámetros	Requerido	Verificado por Ruff/Pylance

> PROMPTS.md (20%)

Criterio	Puntaje
>= 3 prompts con estructura RCTAE completa	20
>= 3 prompts con RCTAE pero sin comprensión línea a línea	10
1-2 prompts o sin estructura	5
No entregado	0

> Commits Git (20%)

Criterio	Puntaje
>= 7 commits con mensajes descriptivos	20
5-6 commits descriptivos	15
4 commits o menos / mensajes vagos ("fix", "update")	5

5.4 Errores frecuentes y cómo corregirlos

Error del alumno	Causa	Solución
`ModuleNotFoundError: No module named 's`	Falta `src/`__init__.py` o no corre pytest desde	Verificar `__init__.py` y correr `pytest tests/` desde la raíz de
CI falla en `ruff check` pero local dice OK	Versión de Ruff diferente	Pinear versión en requirements.txt
Tests pasan localmente pero fallan en CI	Dependencia instalada globalmente, no en r	Agregar la dependencia a requirements.txt
`TypeError: 'NoneType' is not subscriptable`	Función devuelve None implícitamente	Verificar que todos los caminos de código tengan `return`
Copilot sugerencia con `from typing import L	Copilot usa código Python 3.8 style	Cambiar a `list[int]` (Python 3.9+)

6. Antipatrones Frecuentes

En el código

Antipatrón	Descripción	Cómo corregir
`if condition == True:`	Redundante	`if condition:`
`if len(lista) == 0:`	Verbose	`if not lista:`
Loop con índice manual	Ver sección 5.2	Usar `enumerate()`

<code>`from typing import List`</code>	Obsoleto en 3.9+	<code>`list[int]`</code>
Función sin docstring	Viola D103	Agregar docstring
<code>`global variable`</code> dentro de función	Mal diseño	Devolver valor y reasignar externamente
<code>`except:`</code> sin tipo de excepción	Captura todo, peligroso	<code>`except ValueError:`</code>

En el uso de IA

Antipatrón	Descripción	Cómo corregir
Copiar código sin entenderlo	El PROMPTS.md no tiene "comprensión"	Exigir sección línea a línea en revisión
Prompt sin contexto	"Dame una función python"	RCTAE completo
Modificar sugerencia de Copilot sin entender	El alumno selecciona pero no entiende	Preguntar: "¿Que hace esta línea?" antes de aceptar

En Git

Antipatrón	Descripción
<code>`git add .`</code> en todos los commits	Commits masivos, difícil de revisar
Mensajes: "fix", "update", "asdf"	No describen ningún cambio
Un solo commit al final	No muestra el proceso de desarrollo
Commit de archivos de compilación	<code>`*.pyc`</code> deben estar en <code>`.gitignore`</code>

7. Preguntas Socráticas Preparadas

Estas preguntas están diseñadas para momentos de pausa en la clase. Tienen respuesta esperada pero no única.

Para T1-A (Modelo Ágil)

P: "El TP2 tiene tests preescritos que definen que debe hacer cada función. ¿Eso es más parecido al modelo cascada o al ágil?"

Respuesta esperada: Es ágil -- tienes iteraciones cortas (commit a commit), feedback inmediato (CI), y la definición del "cliente" (los tests) está clara desde el inicio.

Para T1-B (Referencias)

P: "Si `[a = [1,2,3]]` y `[b = a]`, y luego hago `[b = [4,5,6]]`, ¿que imprime `[print(a)]`?"

Respuesta esperada: `[[1, 2, 3]]` -- reasignar `[b]` no modifica el objeto al que `[a]` apunta. Solo `[b.append()]` o `[b[0] = x]` modificarían el objeto compartido.

Para T1-C (Funciones)

P: "`[es_primo(n)]` implementado con `[for i in range(2, n)]` funciona pero es lento para números grandes. ¿Cómo mejorarlo?"

Respuesta esperada: Solo verificar hasta `[int(n**0.5) + 1]` -- si `n` tiene un factor mayor a `sqrt(n)`, ya tiene uno menor.

Para T2-A (Comprensiones)

P: "`[x for x in range(10) if x % 2 == 0]` vs `[list(filter(lambda x: x%2==0, range(10)))]`. ¿Cuál preferís y por que?"

Respuesta esperada: Depende del contexto, pero en Python la comprensión se considera más "pythónica" para casos simples. `filter` se usa más cuando ya tenemos una función nombrada.

Para T2-B (HOF)

P: "¿Por que importa `@functools.wraps` en un decorador?"

Respuesta esperada: Sin el, la función decorada pierde su `__name__`, `__doc__` y demás atributos. `[sumar.__name__]` devolvería 'wrapper' en lugar de 'sumar', lo que rompe logging, debugging y documentación automática.

Para T2-C (Type hints)

P: "Si Python no valida los type hints en runtime, ¿para que sirven?"

Respuesta esperada: Son documentación ejecutable. Pylance los valida en el editor (feedback inmediato), Ruff puede checkearlos en CI, y mejoran drásticamente la legibilidad para futuros lectores del código.

8. Ruta a los Recursos del Módulo

Artefactos generados

Artefacto	Ruta	Para quien
Filminas (84 slides + portada)	<code>`salida/cursadas/2026/temas/01-diseno-agil-</code>	Docente (para publicar)
Minuta (notas de clase)	<code>`salida/cursadas/2026/temas/01-diseno-agil-</code>	Docente
Guía de estudio del alumno	<code>`salida/cursadas/2026/temas/01-diseno-agil-</code>	Alumnos
Este documento	<code>`salida/cursadas/2026/temas/01-diseno-agil-</code>	Docente

Configuración del módulo

Archivo	Ruta
Diseño del tema (aprobado)	<code>`salida/cursadas/2026/temas/01-diseno-agil-python/disenio.md`</code>
Topic YAML	<code>`salida/cursadas/2026/temas/01-diseno-agil-python/topic.yaml`</code>
Active topic	<code>`_edu/active-topic.yaml`</code>

Material de referencia

Contenido	Ruta
Tutorial Python 3.13 (extraído)	<code>`material/01-conceptos-introductorios/txt/`</code>
Sintaxis de LP (Sebesta + Gabbrielli)	<code>`material/02-sintaxis/txt/`</code>
Paper: Karymsakova (práctica orientada)	<code>`_edu-memory/material/`</code>
Config EDU	<code>`_edu/config.yaml`</code>

Links externos

Recurso	URL
Repo plantilla TP2	<code>`classroom.github.com/a/X4xiTEDQ`</code>
Python 3.13 Tutorial	<code>`docs.python.org/3.13/tutorial/`</code>
What's New 3.13	<code>`docs.python.org/3.13/whatsnew/3.13.html`</code>
PEP 8	<code>`peps.python.org/pep-0008/`</code>
Ruff docs	<code>`docs.astral.sh/ruff/`</code>

Notas finales de clase

Semana 2 -- Antes de T1

- Verificar que el repo plantilla del TP2 este accesible y el Codespace se abre en < 2 minutos
- Tener [python3.13] listo para la demo del REPL (F-16/F-17)
- Preparar un archivo [demo_errores.py] con errores PEP 8 intencionales para la demo de Ruff

Semana 3 -- Antes de T2

- Revisar cuántos alumnos entregaron commits al TP2 (GitLens en el repo)
- Preparar preguntas socráticas según el progreso observado en P1
- Si muchos tienen problemas de imports ([ModuleNotFoundError]): preparar demo de la solución al inicio de T2-B

Al finalizar el módulo

- Guardar los resultados de la retrospectiva de P2-D para ajustar la guía de estudio del Módulo II
- Verificar que todos los alumnos tienen el TP2 aceptado antes del deadline
- El score pedagógico del módulo va en: [salida/cursadas/2026/temas/01-diseno-agil-python/score-pedagogico.md]